Claude Managed Agents API — Claudia Integration Plan

Plan: Claude Managed Agents API Integration into Claudia

Date: 2026-04-08 **Revised:** 2026-04-08 (post-codebase review — 7 commits since initial draft)
Status: Draft — awaiting approval before implementation **Research:** research.md

Codebase Changes Since Initial Draft

Seven commits landed between the initial research and this revision. Key changes that affect the plan:

1. **Declarative inference config** (`data/inference.json`) — Inference routing (SDK agents, model tiers, tools) moved from hardcoded sets to a hot-reloadable JSON file with `watchFile` . The `managedAgents` list should go here, not in a separate env var.
2. **Channel bindings externalized** (`data/bindings.json`) — Hot-reloadable JSON. No plan impact.
3. **Lifecycle hooks** (`src/hooks.ts`) — `before_inference` , `after_inference` , `before_send` , `after_send` with priority ordering. The Managed Agents adapter **must fire these hooks** at the same points as the existing router.
4. **AgentConfig expanded** — New fields: `autoResolver` , `tasks` , `priorityMap` . The `agent-sdk.ts` now builds a 6-section system prompt (persona + heartbeat + tasks + autoResolver + priorityMap + memory + memory rule + dynamic context). The Managed Agents adapter must mirror this exact prompt assembly.
5. **Standing orders pattern** — All agent context files injected as system prompt sections. The Managed Agents agent config must use the same structure at session creation.
6. **Task Flows** (`src/scheduler/task-flow.ts`) — Durable multi-step workflows. Could run inside Managed Agents sessions later, but not in scope for this plan.
7. **SDK upgraded** to `claude-agent-sdk@0.2.97` . Transitive `@anthropic-ai/sdk` is **0.80.0** — this version does **NOT** have `beta.agents` / `beta.sessions` / `beta.environments` . Must upgrade to ^0.86.1 as a direct dependency.

Decisions (from user input)

Question	Answer
Which agents?	`claudia`, `bella`, `marco`, `swarmy`
Session lifecycle?	**Hybrid** — long-lived for active conversations, ephemeral for dispatch/one-shot tasks
Custom tools?	Yes — memory query, KG search, channel message sending
SDK approach?	Upgrade `@anthropic-ai/sdk` directly
MCP for KG?	Yes — expose mcp-memory-pg over HTTPS for native MCP access
Feature flag?	Per-agent flag with pros/cons analysis (see below)
Environment strategy?	Per-agent environments

Feature Flag: Per-Agent `useManagedAgents`

Pros

- **Gradual rollout** — enable one agent at a time, monitor cost/quality before expanding
- **Easy rollback** — flip one flag to revert a single agent without touching others
- **Mixed routing** — some agents stay on Agent SDK (full local tools), others go cloud
- **A/B comparison** — run same prompt through both paths, compare latency/quality/cost
- **Independent lifecycles** — `swarmy` (multi-agent coordination) has different needs than `claudia` (conversational)

Cons

- **Two code paths to maintain** — Agent SDK + Managed Agents adapter, both must handle sessions, errors, timeouts
- **Inconsistent behavior** — users may notice different capabilities between agents (local Bash vs cloud Bash)
- **Config complexity** — registry grows: `useAgentSDK`, `useManagedAgents`, model tier, tool allowlist — many knobs
- **Testing surface** — every agent x path combination needs testing; flag interactions are subtle

Recommendation

Do it. The pros clearly outweigh the cons. The two-path maintenance cost is bounded (both paths share the same session manager, memory pipeline, and post-processing hooks). Start with `swarmy` (lowest risk, most benefit from cloud isolation), then `claudia`, then `bella` / `marco`.

Architecture Overview

```
Channel Adapters (Discord/Telegram/Slack/WhatsApp/Voice/Video)
    |
Dispatcher (per-peer serial, global max 8)
    |
Router (handleMessage)
    +-- resolveAgent() -> agent config (includes useManagedAgents flag)
    +-- resolveSession() -> SQLite (now stores Agent SDK + Managed Agents session IDs)
    |
    +-- [useManagedAgents = true] ----------------------------------------+
    |   |                                                                 |
    |   +-- inferWithManagedAgents()  <- NEW                              |
    |   |   +-- Get/create Managed Agents session (server-side, persistent) |
    |   |   +-- Send user message as SSE event                            |
    |   |   +-- Stream response (handle custom_tool_use for memory/KG)    |
    |   |   +-- On idle -> return response                                |
    |   |   +-- Fallback: inferWithAgentSDK() or inferLocalFallback()     |
    |   |                                                                 |
    |   +-- Session management                                            |
    |       +-- Hybrid lifecycle: active sessions kept alive, idle expire |
    |       +-- Session ID stored in SQLite (same table, new type column) |
    |                                                                     |
    +-- [useAgentSDK = true, useManagedAgents = false] -- existing path   |
    |   +-- inferWithAgentSDK() -> Opus -> Sonnet -> Mac Mini -> Ollama   |
    |                                                                     |
    +-- [both false] -- existing path                                     |
    |   +-- inferWithOpenRouter() -> Mac Mini -> Ollama                   |
    |                                                                     |
    +-- Post-processing (unchanged, works with all paths) ----------------+
        +-- saveConversationMemory()
        +-- maybeExtractAndStoreFacts()
        +-- maybeCompoundResponse()
        +-- registerExchange() + trackTurn()
        +-- trackComplexity() + executeAnticipations()
```

Custom Tools Bridge

Managed Agents sessions can't directly access Claudia's local systems. Custom tools bridge this:

```
Managed Agents Session (Anthropic cloud)
    |
    +-- Built-in tools: bash, read, write, edit, glob, grep, web_search, web_fetch
    |
    +-- Custom tools (client-executed by Claudia):
    |   +-- memory_query    -> queryKnowledgeGraph() -> return formatted context
    |   +-- memory_store    -> storeFacts() -> persist to KG
    |   +-- channel_send    -> sendOutbound() -> send message to channel
    |   +-- wiki_query      -> buildWikiContext() -> return wiki knowledge
    |
    +-- MCP servers (native, via HTTPS):
        +-- mcp-memory-pg   -> direct pgvector access (semantic search, entity CRUD)
```

Per-Agent Environments

Agent	Environment Config	Rationale
`claudia`	Full packages (Node, Python, git), unrestricted networking, GitHub repos mounted	Primary orchestrator — needs full capability
`bella`	Node + creative tools, restricted networking	Creative agent — no need for arbitrary network
`marco`	Node + Python + research tools, unrestricted networking	Research agent — needs web access
`swarmy`	Full packages, unrestricted networking, multi-agent coordination (research preview)	Multi-agent orchestrator — needs everything

Hybrid Session Lifecycle

```
New message arrives for agent with useManagedAgents=true
    |
    +-- Check SQLite for existing Managed Agents session ID
    |   +-- Found -> check session status via API
    |   |   +-- "idle" -> reuse session (send new event)
    |   |   +-- "running" -> wait or queue
    |   |   +-- "terminated" -> create new session
    |   +-- Not found -> create new session
    |
    +-- Session creation:
    |   +-- Use pre-created Agent + Environment (created at startup)
    |   +-- Start session with system prompt + memory context
    |
    +-- Message handling:
    |   +-- Send user.message event
    |   +-- Stream SSE response
    |   +-- Handle custom_tool_use events (memory, channel, etc.)
    |   +-- Collect agent.message text -> return as response
    |
    +-- Session lifecycle:
        +-- Active conversation: keep session alive (reuse across messages)
        +-- 30 min idle: session auto-terminates (Anthropic manages)
        +-- On session drop: consolidate memory (same as Agent SDK path)
```

Cost projection (hybrid):

- Active conversation (5 messages over 10 min): ~$0.013 session-hour + tokens
- Idle between messages: $0 (session auto-terminates after 30min inactivity)
- vs current Agent SDK: $0 session cost (included in Max plan) + VPS resource contention

---

Implementation Phases

Phase 1: Foundation (types, config, SDK upgrade)

Files to modify:

- `package.json` — add `@anthropic-ai/sdk` ^0.86.1 as **direct** dependency (current transitive 0.80.0 lacks Managed Agents API)
- `src/agents/types.ts` — add `useManagedAgents`, `managedAgentId`, `managedEnvironmentId`
- `src/config.ts` — add `ANTHROPIC_API_KEY`, `MANAGED_AGENTS_ENABLED`
- `data/inference.json` — add `"managedAgents": ["swarmy", "claudia", "bella", "marco"]` (follows existing `sdkAgents` pattern, hot-reloadable)

- `src/agents/registry.ts` — read `managedAgents` from `inferenceConfig`, populate flag in `getAgent()`

Types changes:

```
// src/agents/types.ts — current fields for reference:
// name, persona, memory, heartbeat, autoResolver, tasks, priorityMap,
// model, workspaceDir, allowedTools, useAgentSDK

export interface AgentConfig {
  // ... all existing fields ...
  useManagedAgents: boolean; // NEW
  managedAgentId?: string; // NEW: Anthropic-side agent resource ID
  managedEnvironmentId?: string; // NEW: Anthropic-side environment resource ID
}

export interface InferenceResult {
  response: string;
  sessionId?: string;
  tier: "agent-sdk" | "openrouter" | "mac-mini" | "ollama" | "managed-agents"; // NEW
        tier
}
```

inference.json changes:

```
{
  "sdkAgents": ["claudia", "bella", "marco", ...],
  "managedAgents": ["swarmy", "claudia", "bella", "marco"],
  ...
}
```

Registry changes (in `getAgent()`):

```
useManagedAgents: inferenceConfig.managedAgents?.includes(name) ?? false,
```

Phase 2: Managed Agents Adapter

New file: `src/inference/managed-agents.ts`

Core functions:

1. `ensureManagedAgent(agent)` — create or retrieve Anthropic agent resource
2. `ensureManagedEnvironment(agent)` — create or retrieve environment
3. `buildManagedSystemPrompt(agent, message)` — assemble system prompt **identically to `agent-sdk.ts`** (6 sections: persona + heartbeat + tasks + autoResolver + priorityMap + memory + memory rule + dynamic KG/wiki/skills context)
4. `createManagedSession(agent, env, systemPrompt)` — start a new session
5. `sendMessage(sessionId, message)` — send user message, stream SSE response

6. `handleCustomToolUse(event)` — bridge memory_query, memory_store, channel_send, wiki_query
7. `getManagedSessionStatus(sessionId)` — check if idle/running/terminated
8. `terminateManagedSession(sessionId)` — explicit termination

Critical: System prompt parity with Agent SDK. The `agent-sdk.ts` currently builds the system prompt as:

```
## Identity & Persona     → agent.persona (SOUL.md)
## Standing Orders         → agent.heartbeat (HEARTBEAT.md)
## Task List               → agent.tasks (tasks.md)
## Auto-Resolution Policy  → agent.autoResolver (auto-resolver.md)
## Email Priority Map      → agent.priorityMap (priority-map.md)
## Long-Term Memory        → agent.memory (MEMORY.md)
## Memory Rule             → hardcoded instruction
--- (dynamic) ---
{memoryContext}            → KG search + file memory + skills + wiki
```

The Managed Agents adapter must use this exact same structure. Extract a shared `buildSystemPrompt(agent, message)` function that both `agent-sdk.ts` and `managed-agents.ts` call, to prevent drift.

Custom tool definitions (declared in agent config):

```
const CUSTOM_TOOLS = [
  {
    type: "custom",
    name: "memory_query",
    description: "Search Claudia's long-term knowledge graph memory.",
    input_schema: {
      type: "object",
      properties: {
        query: { type: "string", description: "Search query for memory" },
        limit: { type: "number", description: "Max results (default 8)" },
      },
      required: ["query"],
    },
  },
  {
    type: "custom",
    name: "memory_store",
    description: "Store a new fact or observation in the knowledge graph.",
    input_schema: {
      type: "object",
      properties: {
        content: { type: "string", description: "The fact to store" },
        memory_type: {
          type: "string",
          enum: ["factual", "procedural", "preference", "decision"],
        },
        entity_name: {
          type: "string",
          description: "Entity this fact relates to",
        },
      },
      required: ["content", "memory_type"],
    },
  },
  {
    type: "custom",
    name: "channel_send",
    description:
      "Send a message to a Discord, Telegram, Slack, or WhatsApp channel.",
    input_schema: {
      type: "object",
      properties: {
        channel_type: {
          type: "string",
          enum: ["discord", "telegram", "slack", "whatsapp"],
        },
        channel_id: { type: "string", description: "Channel/chat ID" },
        message: { type: "string", description: "Message content" },
      },
      required: ["channel_type", "channel_id", "message"],
    },
  },
  {
    type: "custom",
    name: "wiki_query",
    description:
```

```
      "Search the wiki knowledge base for persistent, curated knowledge.",
    input_schema: {
      type: "object",
      properties: {
        query: { type: "string", description: "Search query" },
      },
      required: ["query"],
    },
  },
];
```

Phase 3: Inference Cascade Integration

Files to modify:

- `src/inference/fallback.ts` — add `inferWithManagedAgents()` path

New cascade for Managed Agents agents:

```
claudia/bella/marco/swarmy (useManagedAgents=true):
  1. Managed Agents API (cloud, persistent sessions)   <- NEW
  2. Agent SDK (Opus/Sonnet, local fallback)            <- existing
  3. Mac Mini MLX                                       <- existing
  4. VPS Ollama                                         <- existing
```

Key change in `fallback.ts`:

```
export async function infer(agent, message, sessionId?, peerId?) {
  // NEW: Managed Agents path (takes priority when enabled)
  if (agent.useManagedAgents && MANAGED_AGENTS_ENABLED) {
    try {
      return await inferWithManagedAgents(agent, message, sessionId, peerId);
    } catch (err) {
      log.warn(
        `[fallback] Managed Agents failed for ${agent.name}, falling back`,
      );
    }
  }

  // Existing paths unchanged
  if (agent.useAgentSDK) {
    return inferWithAgentSDK(agent, message, sessionId);
  }
  return inferWithOpenRouter(agent, message, peerId);
}
```

Hooks integration: The router already calls `runHooks("before_inference", ctx)` and `runHooks("after_inference", ctx)` around the `infer()` call. Since the Managed Agents path is inside `infer()`, hooks fire automatically — no additional work needed. However, the `inferWithManagedAgents()` function must respect `hookCtx.skip` and `hookCtx.modified` the

same way the router does (these are handled at the router level, not inside `infer()`, so this is already correct).

Phase 4: Session Management

Files to modify:

- `src/sessions/db.ts` — add `session_type` column
- `src/sessions/manager.ts` — handle Managed Agents session lifecycle

Schema change:

```
ALTER TABLE sessions ADD COLUMN session_type TEXT DEFAULT 'agent-sdk';
-- session_type: 'agent-sdk' | 'managed-agents'
```

Manager additions:

- `resolveSession()` — check Managed Agents session status via API before returning
- `saveSession()` — store with `session_type = 'managed-agents'`
- `dropSession()` — terminate Managed Agents session via API, then consolidate

Phase 5: MCP Server for Knowledge Graph

Expose mcp-memory-pg over HTTPS so Managed Agents sessions get direct KG access.

Options (pick one):

- A. Add Express route at `/mcp/memory` on Claudia's port 3001 (simplest, reuses existing server)
- B. Deploy mcp-memory-pg as standalone HTTPS service on VPS (more isolated)

Auth: Bearer token (shared secret stored in Managed Agents vault + Claudia .env)

Managed Agents agent config:

```
{
  "mcp_servers": [
    { "name": "memory", "url": "https://claudia.xurman.com/mcp/memory" }
  ]
}
```

Custom tools (`memory_query`, `memory_store`) remain as a simpler interface; MCP gives full entity CRUD.

Phase 6: Dispatch Queue Migration

Files to modify:

- `src/scheduler/dispatch.ts` — option to execute via Managed Agents instead of `claude -p`

When `MANAGED_AGENTS_ENABLED` is true, dispatch creates ephemeral Managed Agents sessions:

- Cloud execution — no VPS resource contention
- GitHub repo mounting — no local clone needed
- Session persistence — long tasks survive VPS restarts
- Better isolation — each task in its own container

Existing `claude -p` path remains as fallback.

Phase 7: Agent Startup Provisioning

Files to modify:

- `src/index.ts` — at startup, create/verify agents and environments

Startup sequence:

1. For each agent with `useManagedAgents=true`:
   - Create or retrieve Agent resource (model + system prompt + tools + MCP)
   - Create or retrieve Environment resource (packages, networking)
   - Store IDs in registry (`managedAgentId` , `managedEnvironmentId`)
2. Log: `[managed-agents] Provisioned 4 agents: claudia, bella, marco, swarmy`

Agents and environments are reusable — created once, referenced by all sessions. Auto-versioned on update.

Rollout Order

Step	Agent	Rationale	Risk
1	`swarmy`	Multi-agent coordination natural fit; lowest traffic; research preview features	Low
2	`marco`	Research agent benefits from cloud web_search and isolation	Low
3	`bella`	Creative agent benefits from cloud isolation for long tasks	Medium
4	`claudia`	Primary orchestrator; most complex, highest traffic	High

Each step: enable flag -> monitor 1 week -> check cost/latency/quality -> proceed or rollback.

Cost Estimate

Agent	Sessions/day	Avg duration	Daily session-hour cost	Monthly
`swarmy`	2	10 min	$0.03	$0.90
`marco`	5	5 min	$0.03	$0.90
`bella`	10	3 min	$0.04	$1.20
`claudia`	30	5 min	$0.20	$6.00
Total				**$9.00/mo**

Plus standard API token costs (same as current). Session-hour overhead is marginal.

Files Summary

File	Action	Phase
`package.json`	Add `@anthropic-ai/sdk` ^0.86.1 as direct dependency	1
`src/agents/types.ts`	Add `useManagedAgents`, `managedAgentId`, `managedEnvId`	1
`src/config.ts`	Add `ANTHROPIC_API_KEY`, `MANAGED_AGENTS_ENABLED`	1
`data/inference.json`	Add `"managedAgents"` array (hot-reloadable, like sdkAgents)	1
`src/agents/registry.ts`	Read `managedAgents` from inferenceConfig, set flag	1
`src/inference/managed-agents.ts`	**NEW** — full adapter (SSE, custom tools, session mgmt)	2
`src/inference/system-prompt.ts`	**NEW** — extract shared prompt builder from agent-sdk.ts	2
`src/inference/agent-sdk.ts`	Refactor to use shared prompt builder	2
`src/inference/fallback.ts`	Add `inferWithManagedAgents()` path	3
`src/sessions/db.ts`	Add `session_type` column	4
`src/sessions/manager.ts`	Managed Agents session lifecycle (status check, terminate)	4
`src/mcp/memory-endpoint.ts`	**NEW** — HTTPS MCP endpoint for KG	5
`src/scheduler/dispatch.ts`	Optional Managed Agents execution path	6
`src/index.ts`	Startup provisioning (create agents + environments)	7
`.env`	Add `ANTHROPIC_API_KEY`, `MANAGED_AGENTS_ENABLED=true`	7

Open Risks

1. **Latency** — Container spin-up may add 2-5s to first message. Mitigation: hybrid lifecycle keeps active sessions warm.
2. **Cost creep** — Unterminated idle sessions accumulate cost. Mitigation: 30min auto-termination + cleanup cron.
3. **Custom tool latency** — Each custom tool call is a cloud-VPS-cloud round-trip. Mitigation: MCP server provides direct access.
4. **Beta stability** — API may change. Mitigation: adapter pattern isolates changes to one file.
5. **Dual session types** — SQLite stores two types with different semantics. Mitigation: `session_type` column + type-aware cleanup.

Success Criteria

- ☐ `swarmy` runs on Managed Agents for 1 week without errors
- ☐ Session reuse works (conversation continues across messages)
- ☐ Custom tools (memory_query, channel_send) work within sessions
- ☐ Fallback to Agent SDK works when Managed Agents is unavailable
- ☐ Cost stays under $15/month for all 4 agents
- ☐ No response latency increase >3s for `claudia`
- ☐ Memory pipeline (fact extraction, consolidation, nudge) works unchanged